Query Optimization

1 The Complexity of Parallel Optimization

Query optimization in parallel databases is significantly more complex than in sequential databases. The search space for execution plans is much larger, and the cost models must account for distributed factors.

Cost Modeling Challenges

A parallel cost model must incorporate:

- **Partitioning Costs:** The overhead of moving data between processors.
- **Skew:** Handling the ”slowest-link” problem where one node delays the entire query.
- **Resource Contention:** Conflict for shared memory, disk I/O, and network bandwidth.

2 Scheduling the Execution Tree

Scheduling involves deciding how to map the logical operator tree to physical resources. The optimizer must decide:

- **Parallelization Degree:** How many processors to allocate to each individual operation.
- **Interoperation Strategy:** Which operations to pipeline, which to execute independently, and which to run sequentially.

The Diminishing Returns of Parallelism

Allocating more processors than optimal can lead to high **communication overhead**. If the time saved by parallelism is smaller than the time spent on coordination, the advantage is negated. Clusters of small operations should often be run sequentially on a single node.

Pipeline Length Constraints

Long pipelines should be avoided. The final operation may spend excessive time waiting for inputs while holding onto precious resources like memory, leading to poor utilization.

3 Optimization Heuristics

Since there are too many plans to explore exhaustively, parallel optimizers use heuristics.

3.1 Teradata Strategy: Broad Parallelism

Teradata systems often adopt a simple but effective heuristic:

- **No Pipelining:** Avoid complex interoperation pipelines.
- **Universal Partitioning:** Parallelize every single operation across *all* available processors.
- **Advantage:** Finding the best plan becomes as simple as sequential optimization, just with a different cost model.

3.2 Volcano Model: The Exchange Operator

Popularized by the Volcano database, this model follows a two-step process:

1. Pick the most efficient **sequential plan** first.
2. Parallelize that plan by inserting **Exchange Operators**.

The Exchange Operator

The exchange operator is a ”special” node in the query plan that handles tuple redistribution. It encapsulates the complexity of partitioning, flow control, and communication, allow standard local operators to work independently in parallel without being ”aware” of the network.

4 Physical Organization

Optimization isn't just about the query; it starts with the **physical storage design**.

- Choosing the right partitioning technique (hash vs. range) is critical to speed up the expected mix of queries.
- The optimal physical organization varies for different query types; the Database Administrator (DBA) must design it for the most frequent workloads.

Parallel query optimization remains a highly active research area as systems scale towards thousands of nodes.